

Object-Oriented Programming

Game Programming is ez, just learn OOP



Lab GIGA IF ITS 2025



TABLE OF CONTENTS

01.

Why OOP?

02.

Class &
Object

03.

Inheritance

04.

Encapsulation

05.

Polymorphism

06.

Abstraction



01.

WHY OOP?

Think about how there are many things
going on in games



Procedural vs Object-Oriented

Procedural Programming:

Paradigma pemrograman di mana sebuah program terdiri dari satu atau lebih prosedur atau fungsi. Dalam pendekatan ini, program akan dieksekusi secara berurutan dari awal hingga akhir, seperti sebuah skenario linear. Setiap prosedur memiliki tugas tertentu dan menerima input serta menghasilkan output.

Object-Oriented Programming:

Paradigma pemrograman di mana program terdiri dari objek-objek yang saling berinteraksi untuk mencapai tujuan tertentu. Setiap objek memiliki atribut dan perilaku yang unik dan dapat berinteraksi dengan objek lain melalui metode yang ditentukan.

Procedural vs Object-Oriented

BANYAK keuntungan OOP dalam GameDev:

1. Pengorganisasian kode yang modular.
2. Efisiensi dan pemeliharaan kode.
3. Lebih cocok untuk sistem game yang complex.
4. Pemanfaatan konsep OOP seperti:
 - Inheritance
 - Encapsulation
 - Polymorphism
 - Abstraction
5. Memudahkan manajemen elemen-elemen pada game

02. Class & Object

Entities = Object



Class dan Object



Class

Class adalah blueprint atau template yang mendefinisikan atribut (data/properti) dan perilaku (fungsi/metode) dari sekelompok objek. Class sendiri tidak menghasilkan sesuatu yang konkret, melainkan hanya kerangka yang menentukan apa yang akan dimiliki oleh objek yang dibuat dari class tersebut.



Object

Object adalah instansi konkret dari sebuah class. Objek memiliki atribut dan perilaku yang ditentukan oleh classnya, dan objek bisa berinteraksi dengan objek lain. Setiap objek dapat memiliki nilai yang berbeda untuk atribut-atributnya, meskipun dibuat dari class yang sama.

Class dan Object



Class dan Object



```
Class: Furina
{
  Int hp;
  Int def;
  Int atk;

  ...
  void attack();
  void skill();
  void burst();
  Void die();

  ...
}
```



```
Class: Klonang
{
  Int hp;
  Int atk;
  void attack();
  void walk();
  void die();
}
```



```
Class: Ghost
{
  Int speed;
  void move();
}
```

Class dan Object

Cara membuat (instansiasi) objek?

```
Furina furina = new Furina();
```

Class Furina
sebagai tipe data

Nama
variabel

Kata kunci
untuk alokasi
memori

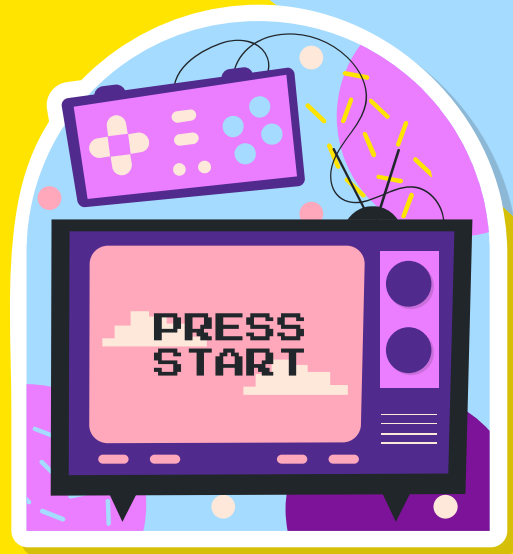
Constructor
untuk class
Furina

```
Furina furina = new Furina(15000, 300, 260);
```

03.

Inheritance

Types of Orc: mage orc, fighter orc, archer orc, and such.



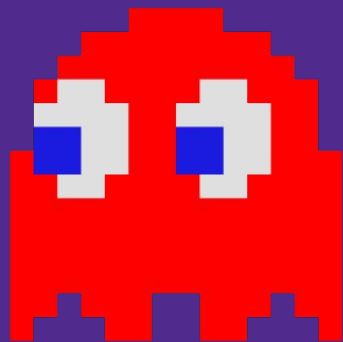
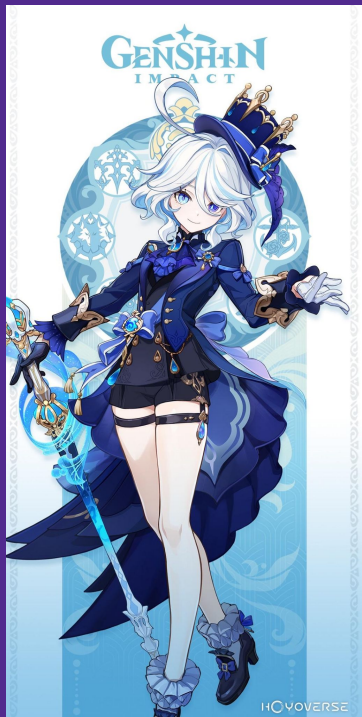
Inheritance

Inheritance memungkinkan sebuah class mewarisi atribut dan metode dari class lain. Konsep ini memungkinkan kita membuat class baru yang berbasis pada class yang sudah ada, dan memperluas atau memodifikasi perilakunya tanpa harus menulis ulang semua kode dari awal.

Inheritance memungkinkan class baru, yang disebut **subclass** atau **child class**, untuk mewarisi semua atribut dan metode dari **parent class** (atau **superclass**).



Inheritance



Inheritance



(Superclass)
Character



(Subclass)
Furina



(Superclass)
Creep



(Subclass)
Klomang

???



Ghost

Inheritance

CHARACTER / NICKNAME	
 - SHADOW	"BLINKY"
 - SPEEDY	"PINKY"
 - BASHFUL	"INKY"
 - POKEY	"CLYDE"

MEREKA BUKAN MERUPAKAN CLASS YANG DITURUNKAN!

Melainkan sebuah objek yang diinstansiasi dari Class Ghost. Hal ini dikarenakan karakteristik mereka yang sama (bisa jalan, dan merupakan musuh)

Contoh:

```
Ghost Blinky = new Ghost("Red");  
Ghost Pinky = new Ghost("Pink");
```

04.

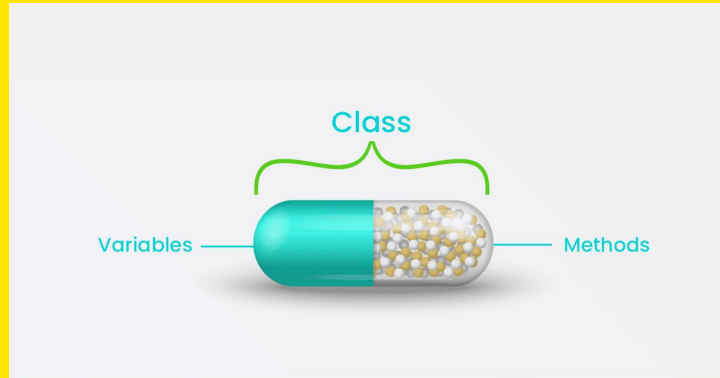
Encapsulation

Objects need privacy too!



Encapsulation

Encapsulation memungkinkan kita untuk menyembunyikan detail implementasi internal sebuah class dari pengguna lain dan hanya menampilkan antarmuka yang diperlukan. Ini dilakukan dengan menjadikan beberapa atribut atau metode private (tidak bisa diakses langsung dari luar class) dan menyediakan public getter dan setter untuk mengontrol bagaimana atribut tersebut dapat diakses atau diubah.



Encapsulation

Player: Memiliki atribut health, kita tidak ingin atribut health ini dimodifikasi secara langsung dari luar, karena hal itu dapat menyebabkan masalah seperti nilai yang tidak valid atau inkonsistensi. Sebagai gantinya, Maka akan dibuat metode **public** untuk mengatur (**setter**) dan mendapatkan (**getter**) nilai health. Lalu memberi access modifier **private** pada health. Contoh:

```
class Character {  
    private int health;  
  
    public int getHealth() {return health;}  
    public void setHealth(int value) {health = value;}  
    public void takeDamage(int damage) {setHealth(health-damage);}  
};
```

05.

Polymorphism

Same instruction, different execution



Polymorphism

Secara harfiah, **polymorphism** berarti "banyak bentuk." Dalam konteks OOP, polymorphism memungkinkan kita mendefinisikan metode dengan nama yang sama dalam class-class yang berbeda, tetapi dengan implementasi yang spesifik untuk setiap class.



Overloading

Compile-time Polymorphism (**Overloading**): Terjadi ketika beberapa metode dalam class yang sama memiliki nama yang sama, tetapi parameter yang berbeda (misalnya, jumlah atau tipe parameternya).



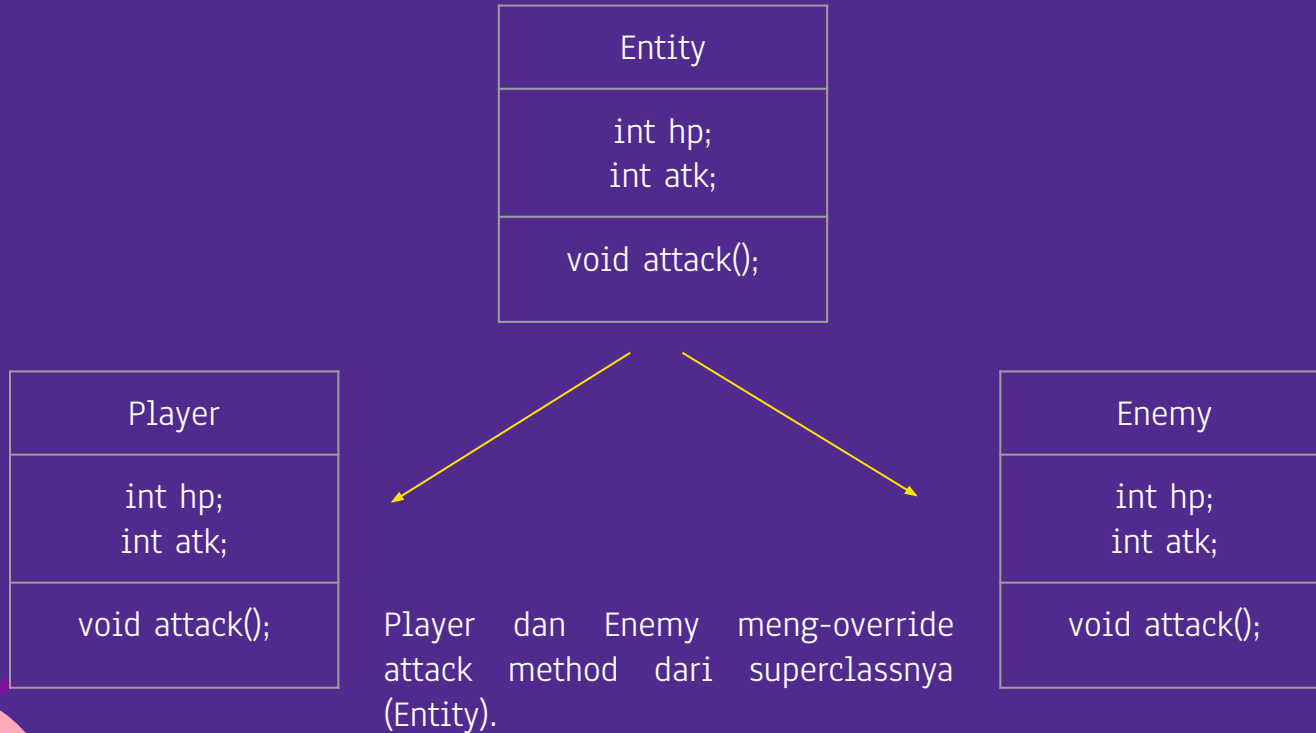
Overriding

Run-time Polymorphism (**Overriding**): Terjadi ketika metode dalam subclass menggantikan atau mengubah perilaku metode dari parent class.

Polymorphism: Overloading

```
class Character {  
public:  
    void takeDamage(int damage) {  
        cout << "Received " << damage << " physical damage." << endl;  
    }  
  
    void takeDamage(int damage, string element) {  
        cout << "Received " << damage << " magical damage of " <<  
        element << " element." << endl;  
    }  
};
```

Polymorphism: Overriding



06.

Abstraction

Things are too complicated! I want it to be simple!



Abstraction

Abstraction adalah konsep dalam OOP yang berfokus pada menyembunyikan detail teknis atau kompleksitas dan hanya menampilkan aspek-aspek penting dari suatu objek atau sistem. Dengan abstraction, kita tidak perlu memahami seluruh detail bagaimana sesuatu bekerja, cukup fokus pada fungsi atau cara penggunaannya.

Dalam OOP, abstraction biasanya diimplementasikan melalui:

Abstract Class: Class yang tidak bisa diinstansiasi dan sering berfungsi sebagai template untuk class lain. Class ini hanya mendefinisikan struktur, tanpa detail implementasi.

Interface: Sekumpulan metode yang harus diimplementasikan oleh class yang mewarisi interface tersebut. Interface mendefinisikan apa yang bisa dilakukan oleh suatu objek, tetapi tidak menjelaskan bagaimana caranya.

Abstraction



Abstraction: Abstract Class

```
abstract class Character
{
    public string Name { get; set; }

    // Abstract method (tidak memiliki
    // implementasi, harus di-override)
    public abstract void Attack();

    // Concrete method (punya implementasi)
    public void Move()
    {
        Console.WriteLine($"{Name} is moving.");
    }
}
```

```
class Hero : Character
{
    public override void Attack()
    {
        Console.WriteLine($"{Name}
attacks with a sword!");
    }

class Enemy : Character
{
    public override void Attack()
    {
        Console.WriteLine($"{Name}
attacks with claws!");
    }
}
```

Abstraction: Interface

```
// Definisi Interface
interface ICharacter
{
    // Interface hanya mendefinisikan kontrak,
    // tidak ada implementasi
    void Attack();
    void Move();
}

// Implementasi Interface pada class Hero
class Hero : ICharacter
{
    public string Name { get; set; }

    public void Attack()
    {
        Console.WriteLine($"{Name} attacks with a sword!");
    }

    public void Move()
    {
        Console.WriteLine($"{Name} is moving.");
    }
}
```

```
// Implementasi Interface pada class Enemy
class Enemy : ICharacter
{
    public string Name { get; set; }

    public void Attack()
    {
        Console.WriteLine($"{Name} attacks
with claws!");
    }

    public void Move()
    {
        Console.WriteLine($"{Name} is
moving.");
    }
}
```

EXTRA CHAPTERS!

Secuile Game Design

What makes a game actually "FUN"?





BONUS CHAPTER: Secuil Game Design

Komponen Utama dalam Game:

- Mekanik Game,
- Tujuan dan objektif,
- Interaktivitas,
- Progression dan rewards,
- Keseimbangan dan kesulitan,
- Feedback,
- Immersion dan naratif.



BONUS CHAPTER: Secuil Game Design

Langkah dalam Game Development:

1. Analysis
2. Concepting
3. Prototyping
4. Development
5. Iteration
6. Assets Design
7. Testing and Balancing
8. Release



THANK YOU!

Now you can literally program any game!